



Nadim Ahmed

Machine Learning in Low Powered Embedded Devices

Metropolia University of Applied Sciences
Bachelor of Engineering
Information and Communication Technology
Bachelor's Thesis
13 April 2026

Abstract

Author:	Nadim Ahmed
Title:	Machine Learning in Low Powered Embedded Devices
Number of Pages:	41 pages
Date:	13 April 2026
Degree:	Bachelor of Engineering
Degree Programme:	Information and Communication Technology
Professional Major:	Smart IoT Systems
Supervisors:	Keijo Lämsikunnas, Senior Lecturer

This thesis outlines the design and development of a real-time audio classification system on a low-power embedded system, the Raspberry Pi Pico 2 with a PDM microphone. The aim was to assess the possibility of implementing machine learning models on microcontrollers with resource constraints to recognize environmental sounds. The system combines audio capture, signal processing, feature extraction, and inference on the device into a single embedded workflow.

A digital MEMS microphone records the audio signals and converts them to PCM via an optimized processing pipeline. CMSIS-DSP is used to extract features directly on the microcontroller, using both the Short-Time Fourier Transform (STFT) and Mel-Frequency Cepstral Coefficients (MFCC) to analyze various representations of audio data. Two neural network models are trained and trained on environmental sound datasets and then transferred to microphone-recorded data to readjust the models to the deployment environment. The models are also optimized with post-training quantization and transformed to TensorFlow Lite format to be executed with TensorFlow Lite Micro.

The experimental findings indicate that real-time audio classification can be performed within memory and computing limits of the Raspberry Pi Pico 2. The STFT-based model was more accurate in classifying and faster to infer, whereas the MFCC-based model used less memory and less input size. These results demonstrate the accuracy efficiency tradeoffs and prove that optimized machine learning models can be effectively implemented on low-power embedded systems to create practical audio classification applications.

Keywords: Embedded Machine Learning, Raspberry Pi Pico 2, DSP, Audio Classification

The originality of this thesis has been checked using Turnitin Originality Check service.

Contents

1	Introduction	1
2	Theory	3
2.1	Machine Learning	3
2.1.1	Supervised Learning and Classification	5
2.1.2	Neural Network and Deep Learning	7
2.2	Embedded Machine Learning	11
2.3	TensorFlow for Embedded Systems	14
2.3.1	TensorFlow Lite	15
2.3.2	TensorFlow Lite Micro	16
2.4	Model Optimization Techniques	16
2.4.1	Transfer Learning	17
2.4.2	Quantization	17
2.4.3	Model Pruning	18
2.5	Audio Signal Processing	18
2.5.1	Microphone Signal Acquisition and Processing	19
2.5.2	Short-Time Fourier Transform (STFT)	20
2.5.3	Mel-Frequency Cepstral Coefficients (MFCC)	21
3	Implementation	22
3.1	Research Approach	22
3.2	Hardware Platforms	23
3.3	Software Environment and Tools	24
3.4	Dataset and Feature Extraction	24
3.5	Machine Learning Model Design	25
3.6	Evaluation Procedure	25
3.7	Training Dataset Preparation	26
3.7.1	Data Argumentation	26
3.7.2	Audio Feature Extraction	28
3.8	Model Preparation	31
3.9	Classification App Development	35

4	Test Result and Discussion	38
5	Conclusion	40
	References	42

List of Abbreviations

GPIO:	General Purpose Input/Output
PIO:	Programmable Input/Output
PDM:	Pulse-density modulation
PCM:	Pulse-code modulation
FFT:	Fast Fourier Transformation
STFT:	Short-Time Fourier Transform
MFCC:	Mel-frequency cepstral coefficients
DMA:	Direct Memory Access
DSP:	Digital Signal Processing
KB:	Kilobyte
MS:	Milliseconds
ML:	Machine learning
AI:	Artificial intelligence
ANN:	Artificial neural network
CNN:	Convolutional Neural Network
ReLU:	Rectified Linear Unit
TinyML:	Tiny Machine Learning

CIC: Cascaded Integrator-Comb

FIR: Finite Impulse Response

1 Introduction

Minuscule low-power devices have experienced impressive advances in capability in the past few years. Tasks previously performed using a desktop computer can now be performed on a board that will fit in the palm of a hand. Modern microcontrollers are no longer restricted to mere input output functions, they now can do complex processing, even some forms of machine learning. Such developments have created possibilities in applications where performance in computing is not as important as efficiency, low cost, and compactness.

Audio classification is among the more difficult tasks to be brought to such devices. It is either the ability to identify voice commands, identify a fire alarm, or track the sounds in the environment, the device should be able to listen at all times and then process the information it hears and make decisions in real-time. Concurrently, it has to operate under stringent memory, processing speed, and power consumption constraints. It is not an easy task to find a design that will satisfy all these requirements.

The aim of the study was to design, implement, and test a full embedded audio classification pipeline on the Raspberry Pi Pico 2, including raw audio capture with a PDM microphone to real-time classification with a machine learning model. The work area involved the creation of a PDM-to-PCM signal conversion process, Mel-frequency cepstral coefficients of the audio, and the deployment of an optimized model that can be used in low-power inference. The project was to determine the viability of such systems in real-life applications and to determine the performance dichotomy between accuracy, processing time, and memory usage.

The study was carried out as a project conducted in Metropolia University of Applied Sciences over the summer. The study focused on the Raspberry Pi Pico 2 microcontroller, which has a dual-core ARM Cortex-M33 processor with

a combined frequency up to 150 MHz. In the project, the processing of raw audio data to determine the noise level and the noise source was one of the goals.

Audio classification was just a component of a larger system, and it also included several environmental sensors. Because of this multi-sensor setup, the audio classification operation was allocated to one core of the microcontroller with the remaining core to deal with other sensor related processes. This setup further limited processing resources of signal processing and machine learning inference.

The central research question addressed is: How can machine learning models be optimised for real-time audio classification on low-powered embedded devices such as the Raspberry Pi Pico 2 when using a PDM microphone?

The report is structured in the following way. Section 2 gives the theoretical background of audio signal processing, MFCC feature extraction, and embedded machine learning techniques. Section 3 outlines the process of implementation, optimisation and approaches taken to achieve the performance targets. Section 4 discussed the test results and summarises the findings. Lastly, Section 5 provides the conclusion and gives future work recommendations.

2 Theory

The chapter presents theoretical background that form the basis of the proposed embedded audio classification system. It contains a summary of machine learning and neural networks models, embedded machine learning and TinyML to run on resource-constrained devices. It also introduces the TensorFlow ecosystem, such as TensorFlow Lite and TensorFlow Lite Micro, and model optimization techniques, such as transfer learning and quantization. Also, the basic concepts of audio signal processing (STFT, MFCC, PDM-to-PCM conversion) are presented to facilitate the implementation presented in subsequent chapters.

2.1 Machine Learning

Machine learning creates systems that can learn and become more effective over time without being clearly programmed. The main purpose of machine learning is to create models capable of detecting patterns, predicting or making decisions, using the input data. These models are statistical approaches that learn based on past data and identify complicated relationships between variables and update with the arrival of new data. In contrast to the traditional rule-based programming, where a programmer hard-codes instructions, ML systems are learned through generalization over examples [1][2].

Advances in computational power in data collection and storage and development of powerful algorithms have driven the expansion of ML. Consequently, ML frameworks have become common in multiple areas, such as visual recognition, cybersecurity, language modeling, financial prediction, medical diagnostics, autonomous systems, and speech recognition [3,4]. In its nature, ML attempts to mimic the flexibility of humans, giving the system the capability to improve by experience and adjust to unfamiliar environments with minimal amount of human intervention [5].

In conventional programming, a human expert provides clear rules and commands to a system to adhere to in order to accomplish a task. The task logic is pre-defined and coded in the program manually. An example of this is a programmer creating a script to sort a list or to compute tax according to predetermined rules. Machine learning, on the contrary, is not based on predetermined rules. Rather, the system is fed data (usually labeled) and algorithms are applied to identify the underlying patterns. The model learned using previous data and use to either predict or classify new data.

This paradigm shift allows the ML systems to deal with tasks that are hard or impractical to solve using rule-based systems. Indicatively, the process of recognizing human speech, recognizing objects in images, or comprehending text can often be complex in patterns and variability that cannot be readily described by explicit rules. Machine learning allows computers to form their own internal model of these patterns based on data, which makes them more flexible and perform better in complex settings [4][6]. Figure 1 below shows the conceptual difference between ML and programming.

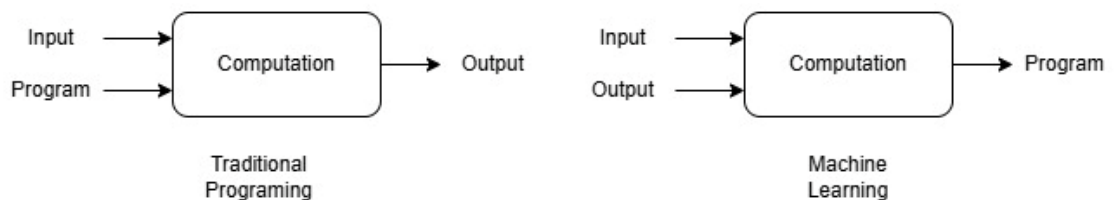


Figure 1: Comparison of Traditional Programming vs Machine Learning Approaches

Figure 1 demonstrates the main conceptual distinction between machine learning and traditional programming. In conventional programming, the program and the input are intertwined to produce the output. Machine learning, on the other hand, is the opposite given examples of inputs and outputs, it produces the program (model) that maps between the two. This graphical illustration emphasizes the way machine learning systems are trained using past data to predict trends that can be used on unknown data.

As it is demonstrated above, machine learning is a paradigm shift in software construction and generalization. The following section will discuss the algorithms that can be used to support supervised learning and classification, which are the basis of real-time audio classification in embedded systems.

2.1.1 Supervised Learning and Classification

Supervised learning represents a fundamental pillar of machine learning, in which models are trained on a dataset that consists of inputs and their outputs. The model does not need to be explicitly programmed, but instead learns through the analysis of these input output pairs and determining the relationship between them. This is to allow the model to extrapolate on this training to predict accurately the results of new, unseen data. This learning is based on exposure to numerous labeled examples, and the system will gradually learn the underlying patterns and relationships. [8][9].

Both classification and regression represent core objectives within supervised learning. Classification is specifically the methodology for mapping input variables into discrete classes or categorical labels. It is more appropriate in those problems where the output variable is discrete, e.g. spam detection, medical diagnosis or audio classification. Regression, however, focuses on predicting continuous numeric values, such as the price of stock or the temperature prediction. In spite of the fact that both tasks are based on the same principles of learning, classification is the more appropriate method to use in this thesis because it is based on the recognition of audio events [7].

In classification tasks, the model learns to distinguish between different classes by analyzing the labeled data and finding the optimal decision boundaries. These models are evaluated based on their ability to generalize correctly predicting labels for previously unseen data. A typical workflow includes partitioning the available data into development and evaluation subsets, training the model using the training partition, and assessing its generalization capabilities on the unseen

set. Various algorithms have been developed to tackle classification problems, each with its own advantages and assumptions. However, selecting an appropriate algorithm is contingent upon the characteristics of the dataset, the dimensionality of the features, interpretability needs, and computational constraints [2]. The general process of supervised learning and classification is illustrated in Figure 2.

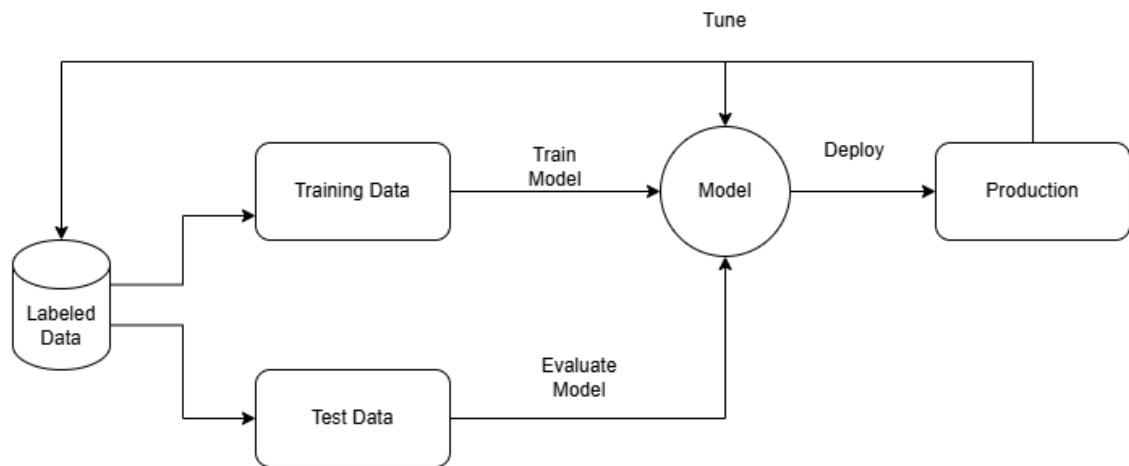


Figure 2: General Workflow of Supervised Learning and Classification [2]

Figure 2 has been modified and adapted as shown above, based on Mahesh [2]. It demonstrates the general flow of a monitored learning system. Raw data is divided into training and validation partitions. Model optimization occurs via the supervised training set, while the test partition is utilized to validate the model's generalization. Depending on these outcomes, the network can be tuned prior to final implementation into production and tuned afterwards depending on performance.

Supervised learning offers a clear structure and a measurable performance, which is why it is appropriate in practical scenarios where there are labeled data. The supervised learning paradigm of classification is applied here to allow embedded devices to recognize and classify environmental audio signals with accuracy. The following paragraphs discuss particular supervised learning

techniques that were implemented in the project, with particular attention to the neural network architecture that was used in this instance.

2.1.2 Neural Network and Deep Learning

Artificial neural networks (ANNs) represent computational frameworks of computation that are based on the architectural layout and operational dynamics of organic neural systems within the human cranium. These models are used to simulate the process of neuron communication to process information, and systems learn through the data without being programmed to handle every situation. A simple neural network is a network of connected nodes, or artificial neurons, arranged in layers: an input layer that accepts data, one or more hidden layers that process data, and an output layer which generates results. The neurons use a weighted sum of the inputs and then an activation function is implemented to introduce non-linear characteristics into the architecture to enable the network to approximate complex functions [10].

The fundamental unit of a neural architecture is the individual node, which resembles biological counterparts because it receives signals, weights them, and sends the output through an activation threshold. Neuron-neuron connections are associated with adjustable weights that are updated throughout the optimization phase to minimize the variance between forecasted and ground-truth results. The process of training is usually done by forward propagating data through the network and backward propagating errors to optimize weights, which is called backpropagation [11]. Neural networks are good at activities that require pattern recognition, including classification and regression, because they possess the capacity to model complex, non-linear dependencies between data. The architectural layout of an artificial neural network and deep neural networks is illustrated in Figure 3.

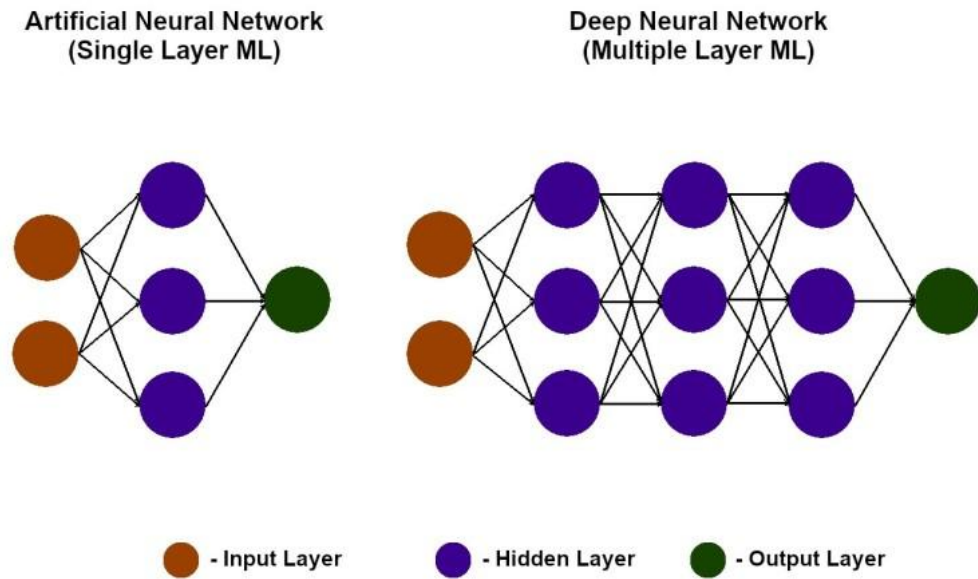


Figure 3. Structure of an Artificial and Deep Neural Network [14]

Figure 3 depicts a side-by-side evaluation of a basic artificial neural model (left) containing one intermediate stratum, versus a deep neural framework (right) featuring numerous internal layers. The entry, intermediate, and terminal tiers are uniquely shaded to illustrate the progressive stream of information.

Deep learning is a more sophisticated form of neural networks, with multiple hidden layers usually dozens or hundreds of layers that permit feature learning in a hierarchy. In opposition to standard shallow networks, deep learning systems can inherently distill hierarchical representations directly from raw sources, bypassing the need for human-led feature extraction. [12]. This richness allows better processing of high-dimensional data, including images or audio, with lower levels identifying simple features and higher levels assembling them into complex ideas.

Convolutional architectures for analyzing grid-like spatial structures and recurrent models for managing time-series information are examples of deep learning architectures that can be trained using this layered method to obtain unmatched results across domains including visual perception or automated linguistic

analysis. Multi-layered network training entails the optimization of large sets of parameters with methods such as gradient descent and can be computationally intensive [13]. Nonetheless, when trained, the models can conduct inference effectively on embedded systems, which is why they are applicable to the implementation in low-power settings.

The shift to deep learning has transformed machine learning by allowing end-to-end learning on raw inputs and lessening the need to use domain knowledge to select features. Practically, deep learning has proven to be significantly more accurate in complex tasks than traditional ones but requires larger datasets and hyperparameter optimization to prevent overfitting [14].

Convolutional Neural Networks (CNNs) represent a specialized class of deep learning frameworks specifically designed for structured grid data, such as images. CNNs are based on the organizational layout of the visual cortex within the human mind and are programmed to automatically identify and learn hierarchical patterns in data in multiple layers of abstraction. In contrast to general-purpose neural networks, CNNs introduce such mechanisms as local connectivity and parameter sharing, which minimize the computational complexity and maximize efficiency in the tasks with spatial hierarchies [15].

The fundamental novelty of CNNs is that they are able to simulate human visual perception, with initial layers detecting basic characteristics such as boundary lines, and consecutive levels merging them into complex objects. This renders CNNs very appropriate to be applied to computer vision challenges, such as photograph classification, multi-object identification, and segmentation. Over the past few years, CNNs have exhibited remarkable results in a wide range of applications where they are able to process raw pixel information without requiring a lot of manual feature engineering [9]. The typical layout of a Convolutional Neural Network is illustrated in Figure 4 below, which depicts the series of operations beginning with the input signal and propagating toward the internal layers to the final classification.

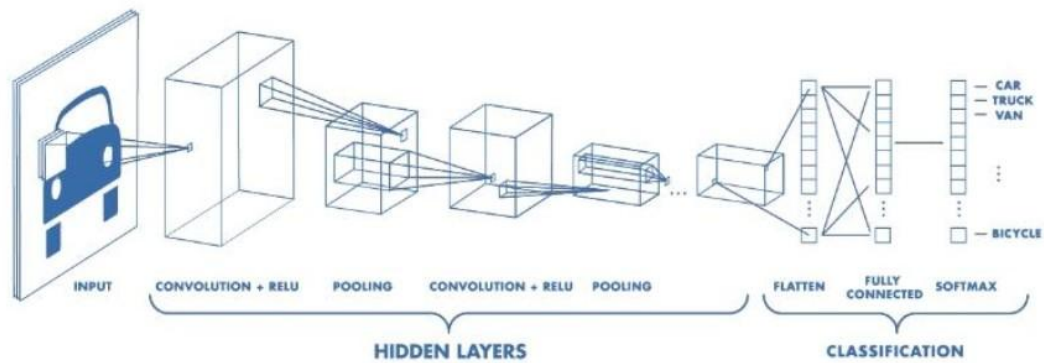


Figure 4: Basic architecture of a Convolutional Neural Network [15]

Figure demonstrating the input stage, feature extraction strata and filters, subsampling tiers, dense layers, and final classification layers; feature map development and convolutional operations. A standard CNN architecture has four main layers: including convolution, subsampling, dense connections, and non-linear (activation) functions.

Convolutional Layer: It serves as the fundamental component of CNNs, which uses learnable filters (kernels) to scan input data and extract features. Every kernel does a convolution, which involves dot products of the weights of the kernel with local areas of the input to generate feature maps that enhance patterns such as edges or textures. Sharing of parameters among the input minimizes the trainable parameters, and the network is efficient with large datasets [15].

Pooling Layer: Pooling is a process that occurs after convolution and which decreases the spatial resolution of feature maps but does not lose important information. Max pooling (selecting the peak magnitude within a localized region) alongside mean pooling are common methods that can be used to address overfitting and computational complexity by summarizing local features [15][9].

Fully Connected Layer: These layers are close to the output and combine all the features of the earlier layers to do high level reasoning, e.g. classification. Every node within this tier maintains links to every activation from the preceding level

and this enables the architecture to correlate derived patterns with final output values [15].

Non-Linear Layer (Activation): Activation functions add non-linearity, which enables CNNs to capture complex relationships. Popular functions are Sigmoid (binary-valued outputs), Tanh (centered around zero), ReLU (rectified linear unit, which trains faster because it avoids vanishing gradients), Leaky ReLU (negative values allowed), Noisy ReLU (regularization via noise), and Parametric ReLU (learnable slope of negatives) [15].

These layers perform sequential convolution feature extraction, pooling downsampling, non-linear activations, and fully connected classification.

2.2 Embedded Machine Learning

Embedded machine learning also known as TinyML is a new area that allows the execution of AI-driven frameworks on incredibly low-resource embedded systems including MCUs and distributed sensors. TinyML, in contrast to traditional cloud-based ML systems, does inference on the device, which is much faster and consumes less energy, as well as enhances privacy and operational independence. This paradigm is especially appealing to large-scale Internet-of-Things (IoT) applications, in which it is neither efficient nor sustainable to transmit all sensor data to the cloud [16][17].

TinyML devices are usually powered by a small number of Kilobytes in RAM alongside milliwatts of energy consumption [18]. Nonetheless, with ongoing progress in lightweight neural networks and model-compression algorithms, nowadays objectives such as wake-word identification, motion classification, and environmental irregularity identification can be performed directly on ultra-low-power hardware [18][19]. TinyML combined with IoT is likely to drive the deployment of distributed intelligence in smart environments, industrial automation, and personalized healthcare [17][19].

The TinyML architecture and workflow is a sequence of interrelated steps that serves to design, refine, and implement machine learning architectures to embedded systems. Alajlan and Ibrahim [19] state that the TinyML workflow consists of seven consecutive steps, each of which plays a role in converting raw sensor data to deployable intelligence on microcontrollers. The general workflow of the TinyML architecture is depicted in Figure 5.

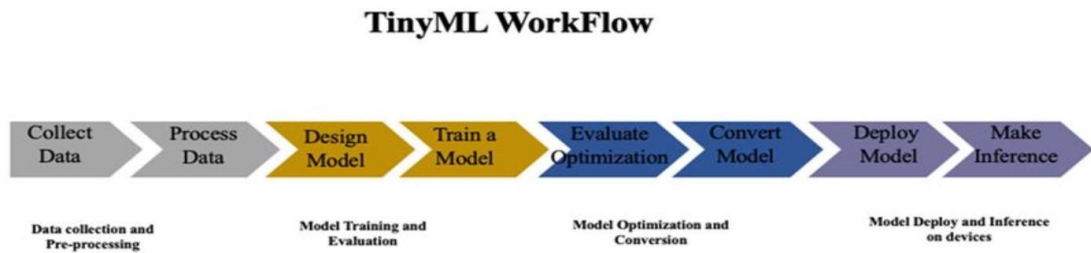


Figure 5: Workflow of TinyML architecture adapted from Alajlan and Ibrahim [19].

As shown in Figure 5, the TinyML workflow comprises seven interrelated stages that together support the creation and implementation of intelligent embedded systems. The process begins by gathering information via various sensing units or IoT devices and then pre-processing where noise is eliminated and important features are extracted to train a model. This architecture is subsequently developed and evaluated using the refined dataset and when good performance is attained it is stored in TensorFlow formats such as .pb or .h5 so as to further optimize it. Model optimization and conversion to quantization methods such as quantization-aware training (QAT) or post-training quantization (PTQ), such as dynamic range (DRQ) and full-integer quantization (FIQ), are then used to reduce computational complexity and memory footprint. The optimized architecture is ported into the TensorFlow Lite environment (TFLite) format to be run efficiently and its performance is measured using the TFLite Interpreter. The model is then transformed into the TensorFlow Lite format for Microcontrollers (TFLM) to turn it into a C array to be easily integrated into C/C++ firmware, which can then be executed on bare-metal IoT devices lacking a traditional OS. After deployment, the model will then make real-time inferences on live sensor data, acting automatically based on the prediction, and keeping track of inference time, flash usage, and RAM consumption to ensure efficiency. The last phase is a fine-tuning

and performance assessment, which is done repeatedly to maintain the robustness and adaptability of the model to changing working conditions [19].

Challenges and Solutions

Several significant obstacles are linked to the implementation of TinyML systems. The first technical limitation is the heterogeneity of hardware as embedded platforms differ in architecture, memory, and the number of peripherals available. This heterogeneity makes cross-platform portability of ML models difficult [16]. Frameworks such as TensorFlow Lite Micro address this with a platform-agnostic interpreter that can execute the same model binary on different hardware with only a few adaptations [16].

The other difficulty is the extreme resource constraints both in memory and computational power. Conventional deep-learning models need to be simplified by forceful optimization. The common method of reducing the model size and energy usage while preserving accuracy is quantization, which transforms floating-point parameters into lower-precision integers, and pruning, which eliminates unnecessary network connections [18][19]. Special hardware accelerators and software co-design strategies can also be used to increase energy efficiency by leveraging low-power instruction sets on contemporary microcontrollers [18].

Long-term accuracy is another challenge, because models used in dynamic environments may degenerate over time. Recent studies on continuous learning show that lightweight on-device adaptation deployed as part of the TensorFlow Lite system can be used to reduce this problem and allow systems to evolve gradually to novel inputs without needing to re-initialize training [20].

Applications of TinyML

TinyML enables a diverse array of real-life scenarios where instantaneous inference, low latency, and energy efficiency are crucial. TinyML can be used in

smart-city infrastructure to enable intelligent traffic management, adaptive street lighting, and environmental monitoring by enabling distributed sensors to make local decisions, instead of using remote servers [19]. In industrial IoT, it is used to facilitate predictive maintenance systems that detect anomalies by analyzing vibration or acoustic signals at the point of location [17]. Wearable technologies and healthcare are no exception, since embedded ML models can process physiological data, e.g. heart rate or motion, in real-time on the device, both providing prompt notifications and protecting privacy [18].

In these areas, TinyML has been known as a disruptive solution that fills the gap between artificial intelligence and embedded computing everywhere. It provides a platform to the next generation of intelligent, connected systems by supporting efficient, secure and autonomous network-edge data processing [20].

2.3 TensorFlow for Embedded Systems

Developed by Google, TensorFlow represents a versatile, open-source AI system that offers a single system to design, train and deploy predictive architectures throughout an extensive range of computing platforms. Originally designed to run on large-scale systems with high performance, TensorFlow has been developed to be used on a broad spectrum of devices, such as cloud servers, smartphones, embedded processors, and more [22]. It is designed in a modular way that enables developers to create models with low-level computational graphs as well as high-level APIs like Keras, and provides visualization, model optimization, and hardware acceleration tools. This has made TensorFlow one of the most popular frameworks within research circles and commercial sectors alike.

As the need to use artificial intelligence on edge and embedded devices has increased, TensorFlow has been modified to fit within the harsh constraints of low-power hardware. This development has seen the emergence of TensorFlow Lite (TFLite) and TensorFlow Lite Micro (TFLM) lightweight frameworks that can be used to perform efficient inference on the device. TensorFlow Lite is an optimized code to run on mobile and embedded Linux environments, and

TensorFlow Lite Micro is further optimized to run on microcontrollers with no operating system [16]. Combined, these frameworks allow developers to execute small, quantized, and hardware-efficient models that are highly accurate and have minimal computational and memory needs. This scalability and efficiency has made TensorFlow a foundation technology of embedded artificial intelligence, balancing the high-level development of machine learning and the ultra-low-power, real-world deployment [17].

2.3.1 TensorFlow Lite

TensorFlow Lite (TFLite) provides a compact implementation of the TensorFlow platform that allows the integration of predictive architectures into mobile, embedded, and edge devices. It solves the issues of traditional ML systems, which are generally too large and resource-demanding to run on low-power hardware. TFLite is a workflow with a start point of model training on high-performance systems with standard TensorFlow, and a finish point of transforming the developed architecture to a hardware-friendly format (.tflite) using the TensorFlow Lite Converter [1]. The format is in the form of tflite which uses a FlatBuffer representation that reduces the parsing overhead and enables rapid loading and execution on devices with limited memory. After conversion, models are optimized with methods such as quantization and pruning to lower the computational requirements and enhance inference speed without a severe loss of accuracy [18].

The TFLite architecture has an Interpreter element that is responsible for executing the architecture on local hardware with pre-optimized kernels. It also provides CPU-based inference as well as hardware acceleration through GPU, DSP, and NPU delegates. This flexibility enables developers to tradeoff between performance and energy efficiency with respect to the deployment platform capabilities. TFLite is used in edge applications to perform real-time inference on-the-fly, removing the dependency on remote computing, minimizing delay, while preserving information security. Quantized models translated into TensorFlow Lite have shown significant memory and power savings, and similar accuracy to

their floating-point equivalents, as shown in experimental implementations, and have been shown to be useful in TinyML and IoT applications [20].

2.3.2 TensorFlow Lite Micro

The TensorFlow Lite Micro (TFLM) framework represents a very thin implementation of TensorFlow Lite developed specifically for executing predictive models on microcontrollers as well as deeply integrated systems. It allows real-time inference using hardware with very constrained storage and processing capabilities, often operating independently of any dynamic memory management. Unlike the conventional ML models, TFLM allocates memory statically by a fixed tensor arena, which guarantees predictable performance and eliminates memory fragmentation at runtime. Models that have been trained and converted to TensorFlow Lite are embedded in the device firmware as C arrays, and they can be executed with just minimal Kilobytes of memory and a small amount of non-volatile space [16].

The modular design and portability of TFLM allow the library to work across a large range of hardware architectures on which TinyML is applied. It offers a standardized API that hides hardware variations and also allows platform-specific optimizations with vendor libraries, like Arm CMSIS-NN. This is because the same model can be deployed across various microcontroller architectures easily. TensorFlow Lite Micro has been a core component of the TinyML framework, enabling applications such as keyword detection, anomaly detection, and environmental sensing to be run on-device with low-latency and consuming minimal energy, without relying on the cloud [16][17].

2.4 Model Optimization Techniques

Resource-constrained embedded and edge devices are important for the implementation of neural network models through architectural optimization. Within the framework of TinyML, optimization methods are aimed at minimizing model size, computational complexity, and energy usage without compromising

on reasonable accuracy and inference time. These techniques facilitate effective execution of machine learning models in hardware systems that have low memory, processing and storage capabilities. The most frequently used optimization techniques are transfer learning, quantization, and model pruning, which deal with various facets of computational and storage efficiency [17][18].

2.4.1 Transfer Learning

Transfer learning allows the reusing and optimization of a previously developed model which is typically developed using massive data repositories, for a fresh yet interconnected objective. The approach greatly minimizes the requirement of large, labeled data and computing power in training, which is suitable in embedded and on-device applications. Practically, the retraining of only the last layers (the "head") of a model is done, and the previous layers (the "base") are kept fixed, preserving general learned features in the source domain [21]. Transfer learning can be used in TensorFlow Lite to enable personalization and adaptation of models locally on-unit and does not require the transmission of data externally to improve privacy, adaptability, and response time. Nevertheless, issues such as catastrophic forgetting, where the models forget previous knowledge during the learning of new tasks, restrict its performance in continuous learning in embedded systems [20].

2.4.2 Quantization

A widely used compression approach for TinyML applications. It makes model parameters and activations less precise, often reducing floating-point values to integers that reduces the size of the model and the amount of computation needed. The result of this process is the acceleration of inference, the reduction of memory consumption, and the decrease of energy consumption, which are essential advantages of microcontrollers and IoT devices. Training can be quantized using quantization-aware training (QAT) which emulates reduced-precision arithmetic to reduce the loss of model performance, or after training with post-training quantization (PTQ) to quantizes models without additional training.

Dynamic range quantization (DRQ) and full integer quantization (FIQ) are the significant PTQ techniques. Such techniques, in combination with the conversion workflow of TensorFlow Lite, can be used to perform inference on low-power AI hardware efficiently [19].

2.4.3 Model Pruning

Model pruning is an attempt to remove unnecessary computations and storage from a neural network by eliminating redundant weights, hidden layers, or whole filters. Pruning produces sparser networks which use fewer operations and less memory to infer, by eliminating parameters that produce insignificant effects on the output. It is especially efficient with convolutional and dense layers which usually occupy most of the model parameters. Pruning may be used either in training (structured pruning) or post-training (post-training pruning), commonly preceded by fine-tuning to restore any accuracy lost [17]. Pruning, when used together with quantization, can also create small, power-efficient models that can be used in real-time TinyML systems, allowing longer battery life and faster inference in embedded systems [18].

2.5 Audio Signal Processing

This chapter introduces the fundamental audio signal processing techniques used in the project. It covers the acquisition of audio signals from MEMS microphones, the conversion from PDM to PCM format, and key feature extraction methods such as the Short-Time Fourier Transform (STFT) and Mel-Frequency Cepstral Coefficients (MFCC). These techniques form the basis for subsequent noise level estimation and noise source identification on the Raspberry Pi Pico 2.

2.5.1 Microphone Signal Acquisition and Processing

The micro-electro-mechanical systems (MEMS) microphones are the most common microphones used in modern embedded audio systems because of their small size, high sensitivity, and low power consumption. MEMS microphones are pressure-sensitive diaphragms that have on-chip electronics, allowing direct digital output without an external analog-to-digital converter. These microphones are suitable in embedded and real-time audio processing systems since they offer stable frequency response and can be readily incorporated into FPGA or microcontroller systems [23].

The vast majority of digital MEMS microphones produce a Pulse-Density Modulated (PDM) signal, a one-bit digital signal representing the analog sound pressure. In PDM, the signal amplitude is coded by the number of logic-high pulses in a given time window such that the more 1s in a time window, the higher the pressure and the more 0s in a time window, the lower the pressure. A sigma-delta modulator is normally used to produce the signal, over-sampling the input at a few megahertz and moving quantization noise to higher frequencies [25]. Although this makes PDM easy to create, it cannot be directly analyzed or played back. Thus, the PDM stream needs to be transformed into Pulse-Code Modulation (PCM), which is the amplitude of the audio waveform as multi-bit samples at standard audio frequencies (e.g., 16 kHz-48 kHz). The traditional digital signal processing (DSP) or machine learning algorithms are more convenient to process, store, and analyze PCM data [24].

PDM to PCM conversion consists of a decimation and filtering operation to downsample the high sampling rate of the PDM bitstream and remove high-frequency quantization noise. The conversion process is typically carried out in three stages: a CIC filter together with a Finite Impulse Response (FIR) compensation filter, followed by a final down-sampling or low-pass operation. CIC filter is also a large scale decimation filter that can be implemented without multipliers and is therefore suitable in hardware and low-power embedded applications [23][25]. Practically, the sampled PDM signal at frequencies of the

order of 1-3 MHz is decimated to tens of kilohertz with a multi-stage CIC filter. But CIC filters are non-flat frequency response filters, and therefore introduce passband distortion. To rectify this, a compensation FIR filter is used to flatten the passband and further reduce out-of-band components [23][25]. In contemporary applications, including those using FPGA architecture, CIC and FIR filtering are used to produce high quality PCM signals with low latency and power consumption [24].

2.5.2 Short-Time Fourier Transform (STFT)

The STFT method is a common signal processing method in audio signal processing that describes the way how frequency components evolve within a signal with time. Unlike the normal Fourier Transform, which assumes stationarity and only gives global frequency information, STFT uses a sliding window over the time domain, which allows localized time–frequency analysis for signals that change over time, such as speech or environmental sounds. STFT can extract transient effects and periodic patterns of the audio signal that are needed in speech recognition, environmental sound classification, and audio synthesis by overlapping frames and performing a FT on each segment [26][27]. There is a balance between time and spectral detail is a matter of the window length used, longer windows are better at frequency accuracy, whereas shorter windows are better at time localization [28].

Practically, STFT generates a two-dimensional image called a spectrogram in time is represented along the horizontal axis, frequency along the vertical axis, and variations in color intensity reflect the signal amplitude. A spectrogram visually summarizes how acoustic energy varies across both time and frequency domains, which is useful in extracting features and machine learning. The choice of window functions, including Hanning or Hamming windows, is used to reduce spectral leakage and enhance accuracy. It has been demonstrated that STFT pre-processing followed by neural network-based processing is more accurate in recognizing sounds in complex acoustic conditions than raw waveforms [30]. This

renders STFT an inevitable process in embedded audio systems to generate features efficiently with limited resources.

2.5.3 Mel-Frequency Cepstral Coefficients (MFCC)

Mel-Frequency Cepstral Coefficients are highly popular characteristics in audio and speech signal. They give a small model of the overall shape of the sound spectrum in a way that closely matches with the human hearing system. It relies on the mel scale, frequency scale that better reflects how humans hear pitch differences, where small changes are more noticeable at lower frequencies than at higher ones [30]. MFCCs highlight perceptually important spectral features and reject redundant information by mapping frequencies using this nonlinear scale, and are widely used in speech recognition tasks, audio classification, and environmental sound analysis [27].

Calculation of MFCCs follows a number of steps. The raw audio split into short-time overlapping frames and then this are subjected to a STFT to get the magnitude spectrum of the frame. The magnitude values are then subjected to a Mel filterbank which is a collection of triangular bandpass filters uniformly distributed across the mel scale. The energy in every band is calculated then converted to a logarithmic scale to simulate the nonlinear sensitivity of human hearing to loudness. Lastly, Discrete Cosine Transform is applied and generate few coefficients, usually the first 12 or 13 coefficients that reflect the spectral envelope of the signal [29][30]. The initial coefficients represent the overall spectral structure, whereas the higher coefficients are frequently fine spectral features or noise, which may be removed depending on the use.

MFCCs are very well adapted to embedded systems and TinyML applications since they significantly decrease the size of the input feature vector and preserve the major discriminative features needed to classify. MFCCs are much simpler to compute and consume less memory than direct processing of spectrograms or raw waveform inputs, which allows real-time inference on low-power microcontrollers [27][30]. Therefore, MFCCs have continued to be a conventional

feature extraction technique in on-device machine learning applications such as keyword spotting, emotion recognition, and acoustic scene recognition.

3 Implementation

This chapter describes the practical implementation of the audio classification system on the Raspberry Pi Pico 2 microcontroller. It covers the research approach, hardware platform, software environment, dataset preparation, feature extraction methods, machine learning model design, optimization for embedded deployment, and the development of the real-time classification application.

3.1 Research Approach

The study was conducted in an iterative design-build-test approach that is usually applied in projects with engineering orientation. The subsystems such as audio capture, signal processing and classification were developed and tested individually. The results of each phase were observed and the hardware setup as well as the software implementation were refined. This was to identify the impact of various feature extraction techniques on the accuracy of classification and computational efficiency when there are stringent hardware constraints. In order to accomplish this, two models were formulated and compared. The former model applied the STFT to produce logarithmic magnitude spectrograms, whereas the latter model applied the MFCC to derive perceptually significant characteristics at a lower dimension. Both models were trained using the same dataset while also implemented on identical same embedded hardware to provide the same basis on which to compare them. The assessment was based on three key measures: classification accuracy, inference time and memory footprint.

3.2 Hardware Platforms

It was implemented on the Raspberry Pi Pico 2 microcontroller board, which is a microcontroller based on the RP2350 dual-core ARM Cortex-M33 processor. The cores are capable of single-precision floating-point operations and run at up to 150 MHz. The device has 512 kilobytes of SRAM and 4 megabytes of external flash memory, which offers a small but adequate amount of memory to lightweight machine learning applications. Figure 6 illustrates The Raspberry Pi Pico 2 development board that was used in this project.



Figure 6: Raspberry Pi Pico 2 W development board used for real-time audio classification.

As illustrated in Figure 6, Pico 2 has 26 GPIO pins and it also provides programmable input/output (PIO) state machines which can be programmed to capture high speed data. Only two pins were used for collecting microphone data.

An Infineon IM69D130 digital PDM microphone was used as the audio input. One of the PIO blocks was connected to the microphone and produced the clock signal and sampled the incoming bitstream. Direct Memory Access (DMA) was used to transfer the raw PDM data to memory to reduce the CPU load.

The microphone was set to record at 16 kHz. The PDM bitstream was converted to 16-bit PCM samples using PDM to PCM filter. The PCM data was then saved in a circular buffer to be processed further to extract features. This design enabled continuous data capture and enabled one processor core to do classification and other sensor related activities at the same time.

3.3 Software Environment and Tools

The entire embedded software was written in C and C++ based on the official Raspberry Pi Pico Software Development Kit (SDK). It was developed with the CMake system and FreeRTOS was used as the real-time operating system to coordinate and schedule tasks and synchronize the activities of multiple tasks running simultaneously. The audio acquisition, feature extraction, and machine learning inference were all run as individual FreeRTOS tasks to provide deterministic execution.

Signal processing functions such as Fast Fourier transform (FFT), windowing, Mel-filterbank computation and MFCC extraction were performed with the Arm CMSIS-DSP library. The highly optimized routines enabled the code to run in real-time and be portable across ARM-based platforms. A desktop computer was used to develop, train, and evaluate the models in Python, TensorFlow, and NumPy. The trained models were then converted and quantized with TensorFlow Lite for Microcontrollers (TFLM) to produce integer-only inference graphs that could be deployed to embedded systems with limited memory and no operating system.

3.4 Dataset and Feature Extraction

The data was represented by one-second audio files with voice commands and environmental sounds. Background noise and fire alarm sounds were also added to enhance diversity and to enhance resistance to real-world conditions. All the recordings were saved in 16-bit mono PCM files with a 16 kHz sampling rate. The audio files were separated into overlapping frames to create time-frequency representations that could be used to classify them.

In the case of the STFT-based model, a window length of 256 samples was utilized, with a 50% frame redundancy, resulting in a 128-sample hop interval. Hamming window was used to minimize spectral leakage and the FFT magnitude was transformed to a logarithmic scale to obtain the final

spectrogram. In the case of the MFCC-based model, processing was done on the same audio frames with a 256 point FFT, and then a bank of 20 triangular Mel filters. In every filtered spectrum, 13 MFCC coefficients were calculated to represent the audio frame in a compact manner. These coefficients were scaled and then they were fed into the neural network as inputs. The MFCC calculation was based on the CMSIS-DSP fixed-point Q15 functions to be consistent with the Python reference code.

3.5 Machine Learning Model Design

The STFT and MFCC models were based on the same lightweight neural network architecture so that their computational and memory demands could be directly compared. The model was made of two fully connected layers with rectified linear unit (ReLU) activations and a softmax output layer which generated class probabilities of the audio frame in question. The training was done on a desktop computer with the Adam optimization was employed with an initial step size of 0.001 over a span of 100 training cycles. The trained models were trained to 8-bit integers to fit within the memory of the microcontroller. Quantization has greatly decreased the size of the model and enhanced the speed of inference at a very small cost to accuracy. Once converted, the MFCC-based model took up about 100 kilobytes of flash memory and the STFT-based model took up about 4 kilobytes because of its higher input feature size.

3.6 Evaluation Procedure

Both models were then run directly on the Raspberry Pi Pico 2 after deployment to test real-time performance and memory usage. The same input audio signals were used to evaluate each model so that there was fair comparison. The most important measurements were accuracy, time of inference and RAM and flash memory consumption. The model was tested by comparing the predictions to labeled test samples to get accuracy. The inference time was recorded as the mean time that it took to process one audio frame between feature extraction and classification output. The FreeRTOS heap statistics and the binary size compiled

were used to estimate memory usage. Measure timing with precision of a few milliseconds using pico's hardware timer.

3.7 Training Dataset Preparation

The quality and diversity of training data are crucial to the effectiveness of any machine learning model as explained in Section 2.1.2. In this project, two primary sources of audio data were used, namely, the Kaggle ESC-50 dataset to create the base model and recordings made with a MEMS microphone to implement the final embedded system. The integrated data was filtered to train and test models to classify environmental sounds in a classroom setting. The audio samples were 16-bit PCM recordings of one second, at 16 kHz. In order to have an equal representation of all classes, the raw recordings were pre-processed to eliminate silent areas, equalize the amplitude, and cut off unnecessary parts. The processed data were then separated into training, development, and evaluation partitions following a 60:20:20 percentage distribution. with the balance in classes to reduce bias. This methodical data preparation strategy improved the generalization of models and provided consistent performance on unseen test samples.

3.7.1 Data Argumentation

In order to enhance the variability of the datasets and make the models more resistant to noise conditions in the real world, a variety of data augmentation methods were used on the audio samples. These techniques are used to increase the size of the dataset and minimize overfitting in training. The augmentations applied to this project were the addition of white Gaussian noise, the addition of random silence, and the development of mix-up samples by joining two or more audio recordings. These changes brought realistic variation without changing class labels. Figure 7 depicts the raw audio signals of alarm sound and background noise.



Figure 7: Raw alarm sound (left) and raw background noise (right).

The audio signal of the alarm in the figure has a structured and periodic waveform with distinctly defined high-energy areas, which is indicative of the tonal and repetitive nature of an alarm sound. Conversely, the background noise signal has a more irregular and low amplitude waveform, which is a representation of unstructured ambient noise that is usually found in the environment. These signal peculiarities indicate the acoustic disparities between target and non-target classes utilized in the preparation of the dataset. Figure 8 represents the audio signal of the argument alarm.

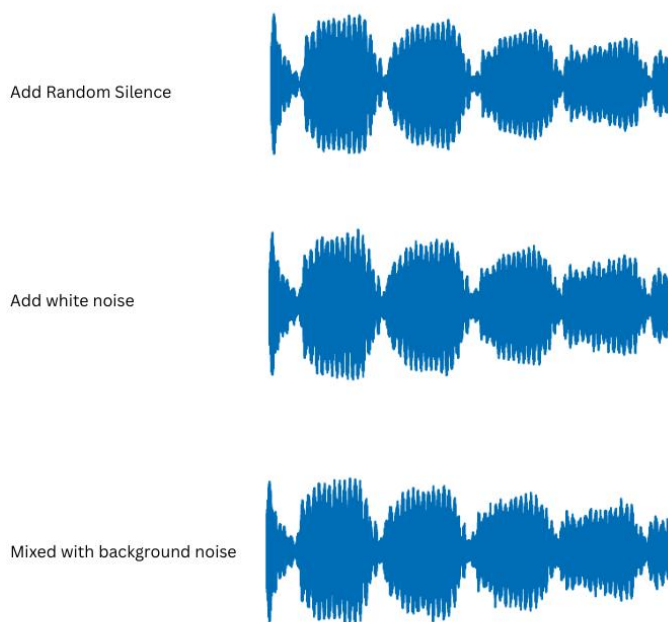


Figure 8: Argument alarm audio signal.

The results of various data augmentation methods used on the alarm audio signal are depicted in the figure. The initial waveform indicates the outcome of the addition of random silence segments that creates temporal gaps and mimics the intermittent sound events. The second waveform is the introduction of white Gaussian noise, which enhances the background interference and models the acoustic disturbances in the real world. The third waveform is the signal of alarm with the background noise, which makes the situation more complicated and acoustically close to the environment. These augmented signals do not change the essence of the original alarm sound, but they add realistic variation, thus enhancing the strength and generalization ability of the trained model.

3.7.2 Audio Feature Extraction

The Raspberry Pi Pico 2 W is based on the RP2350 microcontroller, which has a dedicated hardware subsystem called Programmable I/O (PIO). The PIO is a generalized and deterministic interface, which allows the microcontroller to support custom data protocols and real-time signal processing applications without overloading the main CPU cores. The PIO blocks have several state machines, which can be programmed to run short user-defined programs of instructions to control the GPIO pins directly. This enables developers to use high-speed data interfaces, such as serial communication, pulse-width modulation, or sensor data acquisition, with a small amount of system resources [1].

The PIO is also applicable to the acquisition of Pulse Density Modulation (PDM) data of digital MEMS microphones in the context of audio signal acquisition. Using the PIO state machine to sample the PDM bitstream at a known clock rate, the device can capture data synchronously with accurate timing that would otherwise be hard to accomplish using the standard GPIO polling. The bitstream captured may be then transferred to system memory through Direct Memory Access (DMA) and the data collection may continue at high rate without CPU involvement. This allows the second core or other system tasks to concentrate on digital filtering, PDM-to-PCM conversion and feature extraction.

Signal acquisition with PIO has a lot of benefits to embedded machine learning systems. It guarantees deterministic timing, reduces interrupt overhead, and offers a customizable and low-latency interface that can collect real-time data even on low-power hardware. This is particularly relevant to the TinyML applications, where the effective utilization of scarce computational and energy resources is the key to reliable and continuous on-device audio processing.

The audio samples were processed into feature representations that could be fed into the machine learning models after data augmentation. Two feature extraction techniques were used in this project, STFT and MFCC to build and compare two distinct models based on the accuracy, inference time, and memory consumption on the embedded platform. The two representations provided an opportunity to compare time frequency characteristics with perceptually motivated characteristics in the same classification task.

In both approaches, individual audio signal samples were initially divided into overlapping 32 ms audio frames with a 50 percent overlap. Each frame was windowed to minimize spectral leakage. To achieve the same training and embedded inference behavior, all feature extraction operations were performed with CMSIS-DSP functions, allowing the same numerical behavior on both the desktop training platform and the Raspberry Pi Pico 2.

Figure 9 and figure 10 show the time-frequency representation of the same audio sample processed with STFT and MFCC based feature extraction method respectively to demonstrate the difference between the two methods of feature extraction used in this project.

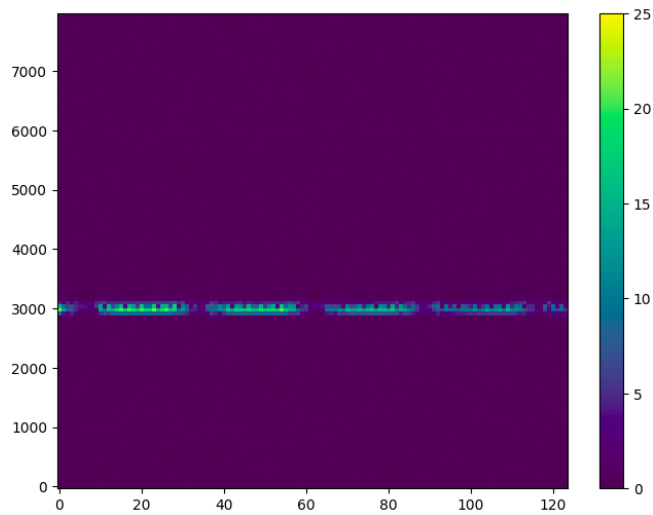


Figure 9: STFT based spectrogram of an audio sample.

The spectrogram obtained with the Short-Time Fourier Transform is presented in Figure 9. Time is plotted on the horizontal axis and frequency on the vertical axis with the intensity of the color being proportional to the signal magnitude. The representation maintains fine-grained frequency content throughout the spectrum and represents temporal changes in acoustic energy. This high time-frequency resolution allows the STFT-based model to acquire localized spectral patterns; but also causes the input representation to be higher-dimensional, which consumes more memory and more computational effort to infer.

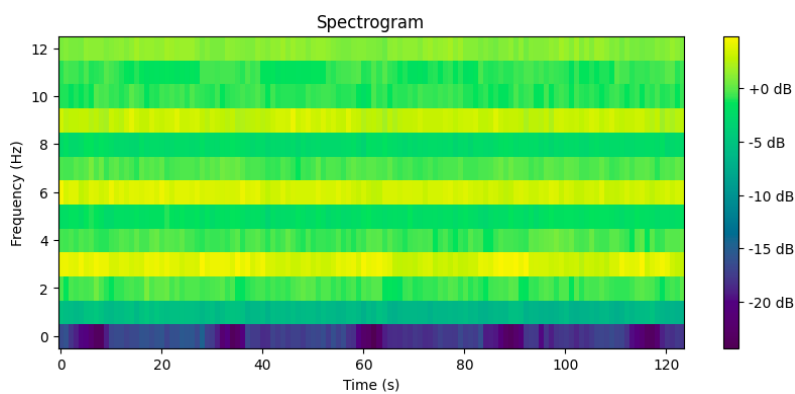


Figure 10: MFCC-based time–frequency representation of the audio sample.

The result of an MFCC-based representation of the same audio signal is presented in figure 10. The MFCC representation, in comparison with the STFT spectrogram, squeezes spectral data by converting frequencies into the Mel scale and only storing a small number of cepstral coefficients. Consequently, the spectral characteristics of the signal that are predominant are retained but the feature dimensionality is greatly reduced. This small-sized representation is especially applicable to embedded systems as it reduces the memory space and inference time and still provides enough discriminative information to classify.

Combined, these numbers point to the inherent trade-off between spectral detail and computational efficiency, on which the comparison between the STFT based and MFCC based models created in this project is based.

3.8 Model Preparation

Section 2.1 discussed the theoretical background of the neural network layers that were used to prepare the base models in detail. On this theoretical basis, practical model architectures were developed and executed on audio classification. Two different models were created: a small base model with STFT features and a bigger experimental model with MFCC features. This methodology made it possible to compare feature representations and model complexity to classification accuracy, inference time, and memory consumption on the embedded platform. Figure 11 shows the underlying architecture of the base model that is ready to accept STFT-based feature input.

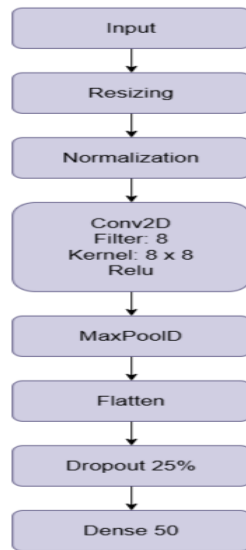


Figure 11: STFT based base model architecture.

Figure 11 represents the STFT based model, which was created as a smaller and computationally efficient architecture. The input of the STFT spectrogram is resized to a constant resolution of 32x32, and then normalized to stabilize training. A single convolutional layer of 8 filter and a large 8x8 kernel is used to extract features and then max pooling is done to reduce dimensionality. The resulting feature map is flattened and regularized by 25 percent dropout, and is directly fed to the 50 output layer to be classified. This small form was chosen to minimize the number of parameters and the cost of inference, which is more appropriate to real-time implementation with embedded constraints. The architecture of the MFCC based model is shown in Figure 12.

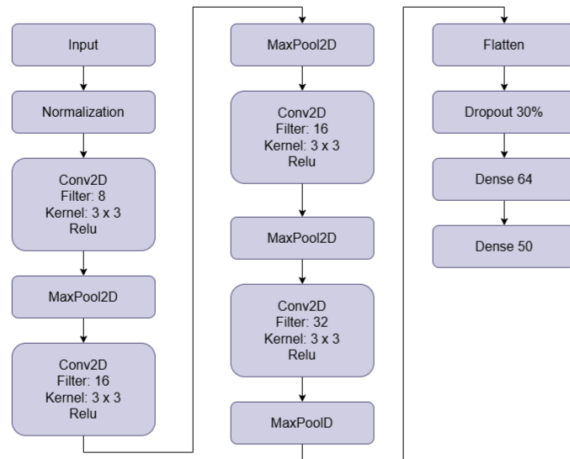


Figure 12: MFCC based base model architecture.

The model based on MFCC as shown in Figure 12 was applied as a bigger experimental network with more representational capacity. The model starts with an input and normalization step and then has several convolutional layers with a successively deeper filter. These convolutional blocks are based on small 3x3 kernels and max-pooling layers to extract hierarchical spectral features whilst steadily decreasing the size of the spatial dimensions. The feature extraction is followed by a flattening followed by 30 percent dropout to regularize the output and then Dense 64 and a final Dense 50 output layer based on the number of classes. This higher architecture was intended to test the hypothesis that MFCC features can be improved by more complex hierarchical feature extraction, even though it would come at the cost of increased computational and memory costs.

Both models were trained with the help of supervised learning and stored as baseline reference models. These trained base models are used as the input in the following part of the implementation where the model sizes and computation complexity are optimized and converted to TensorFlow Lite format. The next section explains the model optimization procedure and conversion to TensorFlow Lite, which will make the models ready to run on the Raspberry Pi Pico 2 with TensorFlow Lite Micro.

Model Optimization and TFLite Conversion

The second step after training the base STFT-based and MFCC-based models was to make them deployable on the Raspberry Pi Pico 2 through reducing training cost, minimizing model size, and compatibility with TensorFlow Lite Micro. Model optimization is a necessity of embedded systems as it is limited by strict memory, computational resource, and power consumption requirements as discussed in Section 2.4.1. This phase involved transfer learning to effectively fine-tune an off-the-shelf base model to the desired task and model optimization and conversion to TensorFlow Lite to fit embedded memory and inference time limits.

Transfer learning was implemented by taking away the original output layer of the base model retaining the other learned layers. Two additional dense output layers were subsequently added and the model was re-trained and fine-tuned with the help of audio data captured on the target microphone. It also allowed the model to adjust to the acoustic properties of the deployment environment by minimizing the number of task-specific training data necessary. The models were then optimized to be embedded and post-trained with quantization, where floating-point weights and activations were quantized to 8-bit signed integer (int8) values to decrease memory footprint and enhance runtime performance. The optimized models were then translated to TensorFlow Lite (.tflite) with the TensorFlow Lite Converter, making sure that all the operators can be inferred on embedded with TensorFlow Lite Micro.

Lastly, the TFLite models were transformed into C arrays to run with TensorFlow Lite Micro. The resulting embedded-ready models were then applied to real-time on-device inference, which is shown in the following section.

3.9 Classification App Development

The classification app combines audio capture, signal processing, feature extraction, and on-board inference into a unified embedded process running on the Raspberry Pi Pico 2. The application was developed to run in real time with tight memory and computational requirements but with a high degree of classification reliability. The implementation of each step of the pipeline was done to make it consistent with the training process and the effective use of the microcontroller resources. Figure 13 shows the application flow of classification.

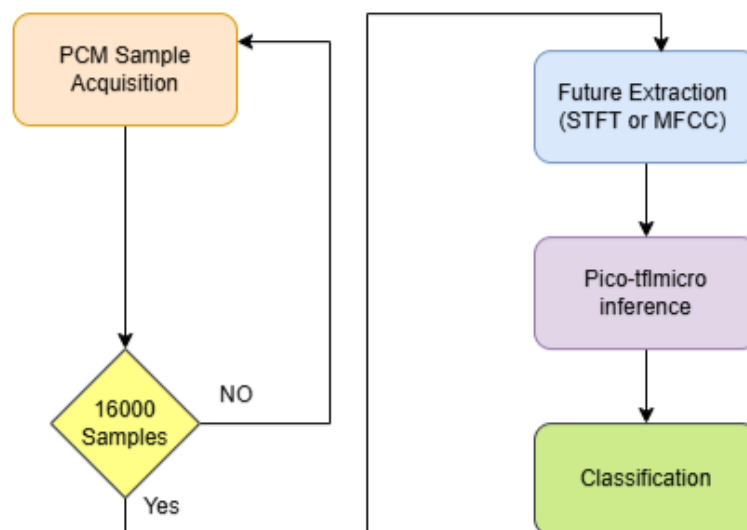


Figure 13: Audio Classification Application Flow.

Figure 13 shows the general flow of the processing of the embedded classification application on the Raspberry Pi Pico 2. The workflow starts with the constant acquisition of PCM samples where audio samples are recorded until a complete one-second frame of 16,000 samples is reached. When the number of samples necessary is achieved, the audio frame is sent to the feature extraction

step where STFT or MFCC features are calculated based on the model chosen. The features extracted are then sent to the TensorFlow Lite Micro inference engine, which does the on-device classification through the optimized neural network model. Lastly, the classification outcome is produced and availed to the application logic to make additional decisions or provide outputs. This sequential flow guarantees deterministic execution and real-time execution within the constraints of embedded systems.

Audio Acquisition and PDM-to-PCM Processing

The audio input was recorded with a digital MEMS microphone which delivers a Pulse PDM bitstream which must be converted to PCM prior to using it to extract features. Figure 14 shows the audio capture and PDM-to-PCM processing chain that runs on the Raspberry Pi Pico 2.

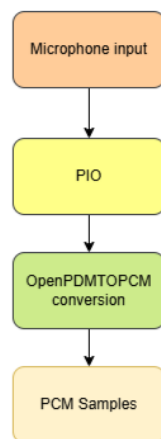


Figure 14: Audio acquisition and PDM-to-PCM processing pipeline.

Figure 14 displays the flowchart of the process of the PDM audio sample from microphone to convert it into the PCM. The Programmable I/O (PIO) subsystem was used to ensure deterministic timing and proper sampling of the microphone clock and data signals. The PIO state machine was programmed to run at 1.072 MHz, and sample the PDM data on the falling edge of the clock signal. The incoming stream of PDM data was sent to memory via 8-bit Direct Memory Access (DMA) and could be acquired continuously without CPU involvement. The

obtained PDM data were transformed to PCM samples with the help of the OpenPDMtoPCM library created by STMicroelectronics. This library was chosen over a custom CIC filter because it is optimized and is applicable to real-time embedded processing. OpenPDMtoPCM uses a CIC filtering step with sinc compensation and look-up tables to effectively decimate and suppress noise at minimum computation cost. The result of this conversion process was 16-bit PCM samples, 16,000 samples were added up per inference window, which was one second of audio at 16 kHz. The resulting PCM signal was a clean and stable input to the later stages of feature extraction.

Feature Extraction Using STFT and MFCC

After the availability of PCM audio frames, the feature extraction was done on the microcontroller itself using CMSIS-DSP functions to guarantee numerical consistency with the training environment. The extraction of features was performed individually based on the model chosen, either STFT or MFCC features. In the case of the STFT-based tests, Short-Time Fourier Transform characteristics were calculated to produce time-frequency spectrograms, which were the input to the STFT-based model. In the case of MFCC-based tests, MFCC features were calculated with CMSIS-DSP routines with precomputed coefficients, such as `dctCoefs`, `filterPos`, `filterLengths`, `filterCoefs` and `windowCoefs`. These parameters were produced in Python when training and directly transferred into the embedded implementation to guarantee the same behavior of feature extraction on both the desktop and microcontroller platform.

TensorFlow Lite Micro Inference

The features were extracted and fed into the TensorFlow Lite Micro inference engine that was running on the Raspberry Pi Pico 2. The quantized and optimized models were loaded into a statically allocated tensor arena to guarantee predictable memory usage. Frame-by-frame inference was undertaken and generated class probability scores of each audio segment. The inference procedure was to be implemented on a real-time basis, and the latency and

memory consumption were to be observed to ensure that it is appropriate to be used in continuous mode.

Output and Decision Logic

The end product of the classification application was the choice of the class with the highest probability score generated by the model. Simple temporal smoothing was used to enhance robustness by averaging predictions over several sequential frames. The result of the classification was then used to invoke application specific actions, including recording events or turning on indicators. The presented end-to-end embedded application confirms the effective implementation of audio signal processing and machine learning inference on a low-power microcontroller platform.

4 Test Result and Discussion

This section compares the performance of the embedded audio classification system implemented as in Section 3. The empirical results contrast the STFT-derived and MFCC-driven architectures against the project goals of real-time execution, effective memory usage, and consistent classification accuracy on the Raspberry Pi Pico 2. The comparison is based on model size, RAM usage, time of digital signal processing, inference latency, and classification accuracy to identify which approach is best suited to low-power embedded implementation. The performance of the STFT and MFCC models running on the Raspberry Pi Pico 2 is shown in Table 1.

Table 1: Performance Comparison of STFT and MFCC Models.

Metric	STFT Model	MFCC Model
Model Size (KB)	4	100
Arena Usage (KB)	17.6	18.13
PCM Size (KB)	31.25	31.25
DSP Time (ms)	54	84
Input Data Usage (KB)	15.62	1.57
Inference Time (ms)	9	63
Model Accuracy (%)	97	85
Total RAM Usage (KB)	64.47	50.95

Table 1 demonstrates that the STFT-based model had a better classification accuracy (97%), than the MFCC-based model (85%). Besides, the STFT model showed a much lower inference time (9 ms) compared to the MFCC model (63 ms). Nevertheless, this better performance demanded more memory resources. The feature representation of the STFT took 15.62 KB of input data and the overall RAM usage was 64.47 KB, compared to 1.57 KB of input features and 50.95 KB of overall RAM usage of the MFCC model.

The MFCC implementation took more DSP processing time (84 ms) than the STFT implementation (54 ms) in terms of signal processing overhead. Even though the MFCC model was much larger in size (100 KB vs. 4 KB), the usage of the tensor arena in both models was also similar, which implies that both models demand similar runtime allocation needs in TensorFlow Lite Micro. Both methods had the same PCM input size because both classifications were made using a one-second 16 kHz, 16-bit audio.

In general, the findings indicate the trade-off of the computational efficiency, memory consumption, and classification accuracy in embedded audio processing systems. The STFT-based model was more accurate and faster to infer whereas the MFCC-based model consumed less memory. These results indicate that optimized machine learning models can be used in the Raspberry Pi Pico 2

hardware, which has hardware constraints, and real-time embedded audio classification is possible.

5 Conclusion

The study examined the possibility of implementing machine learning models to perform real-time audio classification on low-powered embedded computing platforms, namely the Raspberry Pi Pico 2 on a PDM microphone. The main aim was to design, deploy and test a full embedded audio classification pipeline, such as signal acquisition, PDM-to-PCM conversion, signal transformation, network training, hyperparameter tuning, and local inference execution with TensorFlow Lite Micro.

Two feature extraction techniques STFT and MFCC were applied and tested on the basis of convolutional neural network models. The models had been trained in a desktop setting and optimized using transfer learning and post-training quantization to be deployed on the microcontroller. The PIO subsystem and DMA were utilized to achieve audio acquisition with the PDM data capture being efficient and CMSIS-DSP to extract embedded features to maintain consistency between training and inference settings.

The experimental findings proved that the real-time embedded audio classification is possible under the memory and computational limitations of the Raspberry Pi Pico 2. The model based on STFT was more accurate in classification and had shorter inference time whereas the model based on MFCC used less memory and less input data. These findings demonstrate a trade-off between spectral quality, computational efficiency and memory use in developing machine learning systems on a limited embedded platform.

The combination of transfer learning, quantization, effective DSP implementation, and lightweight neural network design was used to answer the research question, which is how machine learning models can be optimized to run on low-powered embedded devices in real time. The paper establishes that, through proper

techniques of optimization and system-level design, embedded microcontrollers are capable of performing machine learning inference to support realistic audio monitoring tasks.

Despite the fact that the system has attained reliable performance, there are limitations. The models were tested in a controlled classroom setting where there were few sound classes. Future directions may include larger datasets, multi-class environmental sound classification, adaptive thresholding methods, or hardware acceleration methods. Energy efficiency and latency performance could also be improved by further optimization of feature extraction and model architecture.

The same model building could be reused to make different machine learning models to perform various types of classification task such as keyword spotting, voice command identification and person identification by changing the window length for digital signal processing. In the microcontroller it has to be updated with new filter coefficients. One of the wisest uses of the sound classification in low powered embedded devices could be anomaly detection in industrial machines and robots.

Altogether, this paper shows the feasibility of embedded machine learning in audio classification and offers a systematic guide to the application of analogous TinyML-based systems on resource-limited microcontrollers.

References

Vancouver (numbering) system:

- 1 Ashmore R, Calinescu R, Paterson C. Assuring the Machine Learning Lifecycle: Desiderata, Methods, and Challenges. 2019. Available from: <https://doi.org/10.48550/arXiv.1905.04223>
- 2 Qiu, J., Wu, Q., Ding, G. et al. A survey of machine learning for big data processing. *EURASIP J. Adv. Signal Process.* 2016, 67 (2016). Available from: <https://doi.org/10.1186/s13634-016-0355-x>
- 3 Woodman RJ, Mangoni AA. A comprehensive review of machine learning algorithms and their application in geriatric medicine: present and future. *Aging Clin Exp Res.* 2023;35(11):2363-2397. Available from: [doi:10.1007/s40520-023-02552-2](https://doi.org/10.1007/s40520-023-02552-2)
- 4 Afifi H, Pochaba S, Boltres A, Laniewski D, Haberer J, Paeleke L, et al. Machine Learning With Computer Networks: Techniques, Datasets, and Models. *IEEE Access.* 2024 Jan 1;12:54673–720
- 5 Yusoff MIM. Machine Learning: An Overview. *Open Journal of Modelling and Simulation.* 2024 Jan 1;12(03):89–99.
- 6 Giray G. A software engineering perspective on engineering machine learning systems: State of the art and challenges. *Journal of Systems and Software.* 2021 Jun 25;180:111031–111031.
- 7 Woodman, R.J., Mangoni, A.A. A comprehensive review of machine learning algorithms and their application in geriatric medicine: present and future. *Aging Clin Exp Res* **35**, 2363–2397 (2023). Available from: <https://doi.org/10.1007/s40520-023-02552-2>
- 8 Mahesh B. Machine Learning Algorithms - A Review. *International Journal of Science and Research (IJSR).* 2020 Jan 5;9(1):381–6.
- 9 Ling Q. Machine learning algorithms review. *Applied and Computational Engineering.* 2023 May 31;4(1):91–8.
- 10 Qamar R, Zardari BA. Artificial Neural Networks: An Overview. *Mesopotamian Journal of Computer Science.* 2023 Aug 2;130–9.
- 11 Janiesch C, Zschech P, Heinrich K. Machine learning and deep learning. *Electronic Markets.* 2021 Apr 8;31(3):685–95.
- 12 Mehmood F, Ahmad S, Whangbo TK. An Efficient Optimization Technique for Training Deep Neural Networks. *Mathematics.* 2023 Mar 10;11(6):1360–1360.

- 13 Taye MM. Understanding of Machine Learning with Deep Learning: Architectures, Workflow, Applications and Future Directions. *Computers*. 2023 Apr 25;12(5):91–91.
- 14 Kufel J, Bargieł-Łączek K, Kocot S, Koźlik M, Bartnikowska W, Janik M, Czogalik Ł, Dudek P, Magiera M, Lis A, et al. What Is Machine Learning, Artificial Neural Networks and Deep Learning?—Examples of Practical Applications in Medicine. *Diagnostics*. 2023; 13(15):2582. Available from: <https://doi.org/10.3390/diagnostics13152582>.
- 15 Purwono P, Ma'arif A, Rahmianar W, Fathurrahman HIK, Frisky AZK, Haq QM ul. Understanding of Convolutional Neural Network (CNN): A Review. *International Journal of Robotics and Control Systems*. 2023 Jan 15;2(4):739–48.
- 16 David R, Duke J, Jain A, Reddi VJ, Jeffries N, Li J, et al. TensorFlow Lite Micro: Embedded Machine Learning on TinyML Systems. *arXiv (Cornell University)*. 2020 Jan 1.
- 17 L. Capogrosso, F. Cunico, D. S. Cheng, F. Fummi and M. Cristani, "A Machine Learning-Oriented Survey on Tiny Machine Learning," in *IEEE Access*, vol. 12, pp. 23406-23426, 2024, Available from: doi:10.1109/ACCESS.2024.3365349.
- 18 Schizas N, Karras A, Karras C, Sioutas S. TinyML for Ultra-Low Power AI and Large Scale IoT Deployments: A Systematic Review. *Future Internet*. 2022 Dec 6;14(12):363–363.
- 19 Alajlan NN, Ibrahim DM. TinyML: Adopting tiny machine learning in smart cities. *Journal of Autonomous Intelligence*. 2024 Jan 26;7(4).
- 20 Demosthenous G, Vassiliades V. Continual Learning on the Edge with TensorFlow Lite. *arXiv (Cornell University)*. 2021 Jan 1
- 21 Baker NA, Zengeler N, Handmann U. A Transfer Learning Evaluation of Deep Neural Networks for Image Classification. *Machine Learning and Knowledge Extraction*. 2022 Jan 14;4(1):22–41.
- 22 Ramchandani M, Khandare H, Singh P, Rajak P, Suryawanshi N, Jangde AS, et al. Survey: Tensorflow in Machine Learning. *Journal of Physics Conference Series*. 2022 May 1;2273(1):012008–012008.
- 23 K. Kowalczyk, S. Wozniak, T. Chyrowicz and R. Rumian, "Embedded system for acquisition and enhancement of audio signals," 2016 Signal Processing: Algorithms, Architectures, Arrangements, and Applications (SPA), Poznan, Poland, 2016, pp. 68-71, Available from: doi: 10.1109/SPA.2016.7763589.
- 24 R. Sum, C. Khongprasongsiri, W. Suwansantisuk and P. Kumhom, "Low Latency PDM-to-PCM Decoder," 2023 20th International Conference on Electrical Engineering/Electronics, Computer, Telecommunications and

- Information Technology (ECTI-CON), Nakhon Phanom, Thailand, 2023, pp. 1-4, Available from: doi: 10.1109/ECTI-CON58255.2023.10153161.
- 25 Liu Y, Lanier J, Fu K, Sample A. PDM-to-PCM Microphone Signal Conversion on FPGA Using CIC and FIR Filters. University of Michigan; 2019. Available from: https://yatian-liu.github.io/public/PDM_PCM_Signal_Conversion_FPGA.pdf
 - 26 Min JH, Gao Z, Wang L, Zhang A. Application Research of Short-Time Fourier Transform in Music Generation Based on the Parallel WaveGan System. IEEE Transactions on Industrial Informatics. 2024 May 15;20(9):10770–8.
 - 27 Gourisaria MK, Agrawal R, Sahni M, Singh PK. Comparative analysis of audio classification with MFCC and STFT features using machine learning techniques. Discover Internet of Things. 2024 Jan 3;4(1).
 - 28 Rosmala D, FADHILAH MN. Audio Conversion for Music Genre Classification Using Short-Time Fourier Transform and Inception V3. ELKOMIKA Jurnal Teknik Energi Elektrik Teknik Telekomunikasi & Teknik Elektronika. 2025 Jan 8;13(1):84–84.
 - 29 S. Davis and P. Mermelstein, "Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences," in IEEE Transactions on Acoustics, Speech, and Signal Processing, vol. 28, no. 4, pp. 357-366, August 1980, doi: 10.1109/TASSP.1980.1163420
 - 30 Tom Bäckström, Okko Räsänen, Abraham Zewoudie, Pablo Pérez Zarazaga, Liisa Koivusalo, Sneha Das, Esteban Gómez Mellado, Mariem Bouafif Mansali, Daniel Ramos, Sudarsana Kadiri, Paavo Alku, and Mohammad Hassan Vali "*Introduction to Speech Processing*", 2nd Edition, 2022. URL: <https://speechprocessingbook.aalto.fi>, DOI: [10.5281/zenodo.6821775](https://doi.org/10.5281/zenodo.6821775).
 - 31 Raspberry Pi Ltd. RP2350 Datasheet – Raspberry Pi Pico 2 Microcontroller Documentation. 2024. Available from: <https://datasheets.raspberrypi.com/rp2350/rp2350-datasheet.pdf>

